LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Gt. Qowita Celia Aurellizha **NIM. 2510817120019**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Gt. Qowita Celia Aurellizha
NIM : 2510817120019

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Pembahasan 3
SOAL 2 8
A. Source Code 10
B. Output Program 11
C. Pembahasan 11
SOAL 3 14
A. Source Code 14
B. Pembahasan 15
SOAL 4 19
A. Source Code 20
B. Output Program 21
C. Pembahasan 21
TAUTAN GIT 23

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2A 11
Gambar 2. Screenshot Hasil Jawaban Soal 4A 21

DAFTAR TABEL

Table 1. Source Code stack.h 1
Table 2. Source Code stack.cpp 1
Table 3. Source Code prob1.cpp 10
Table 4. Source Code queue.h 14
Table 5. Source Code queue.cpp 14
Table 6. Source Code prob2.cpp 20

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1. Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

class stack
{
private:
    int top;
    int arr[100];

public:
    stack();
    void push(int x);
    int pop();
    int peek();
    bool isEmpty();
};

#endif
```

Table 2. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>
```

```

stack::stack()
{
    top = -1;
}

void stack::push(int x)
{
    if (top == 99)
    {
        throw std::overflow_error("Stack Penuh");
    }
    arr[++top] = x;
}

int stack::pop()
{
    if (isEmpty())
    {
        throw std::underflow_error("Stack Kosong");
    }
    return arr[top--];
}

int stack::peek()
{
    if (isEmpty())
    {
        throw std::underflow_error("Stack Kosong");
    }
    return arr[top];
}

bool stack::isEmpty()
{
    return top == -1;
}
```

B. Pembahasan

Stack adalah struktur data yang menggunakan prinsip:

- LIFO (Last In First Out) yaitu, elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan.
- Implementasi digunakan menggunakan array dengan variabel `top` sebagai penunjuk posisi elemen paling atas.

Pada code `Stack::Stack() {`

```
    top = -1;
```

`}` terdapat Constructor stack() yang digunakan sebagai berikut:

- Constructor digunakan untuk inisialisasi stack.
    - `top = -1` menandakan bahwa stack masih kosong
    - Karena belum ada elemen yang masuk.
- Alurnya jika:
    - Saat objek stack dibuat
    - Nilai `top` di set ke `-1`.
- Jika stack kosong, maka tidak ada indeks yang terisi maka nilai awal yang benar adalah `-1`.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Pada code `void Stack::push(int x) {`

```
    if (top == 99) {
```

```
        throw std::overflow_error("Stack penuh");
    }
    arr[++top] = x;
```

`}` terdapat Fungsi push(int x) yang digunakan sebagai berikut:

- Fungsi ini digunakan untuk menambahkan elemen ke dalam stack.
- Alurnya jika:
  - Cek apakah stack penuh `(top == 99)`
  - Jika penuh maka throw error
  - Jika tidak
    - `top` dinaikkan `(++top)`
    - Nilai `x` disimpan ke `arr[top]`.
- Kenapa menggunakan `(++top)` terlebih dahulu, karena posisi awal kosong di `-1` jadi elemen pertama harus masuk ke index `0`.
- Elemen selalu masuk di posisi paling atas `top`.
- Tidak ada elemen lain yang dilewati.
- Jadi urutan stack tetap terjaga sesuai dengan LIFO.
- Kompleksitas:
  - Waktu: **O(1)** (langsung disimpan)
  - Ruang: **O(1)**.

Pada code `int Stack::pop() {`

```
    if (isEmpty()) {
        throw std::underflow_error("Stack kosong");
    }
    return arr[top--];
```

`}` terdapat Fungsi pop() yang digunakan sebagai berikut:

- Fungsi ini digunakan untuk mengambil dan menghapus elemen paling atas.
- Alurnya jika:
  - Cek apakah stack kosong
  - Jika kosong maka throw error
  - Jika tidak
    - Ambil nilai `arr[top]`.
    - Turunkan `top (top--)`.
    - Kembalikan nilai tersebut.
- Karena stack LIFO, elemen yang diambil harus yang terakhir masuk
- `top` selalu menunjuk elemen terakhir.
- Maka `arr[top]` adalah elemen yang benar untuk di-pop.
- Kompleksitas:
  - Waktu: **O(1)** (langsung disimpan)
  - Ruang: **O(1)**.

Pada code `int Stack::peek() {`

```
    if (isEmpty()) {
        throw std::underflow_error("Stack kosong");
    }
    return arr[top];
```

`}` terdapat Fungsi peek() yang digunakan sebagai berikut:

- Fungsi ini digunakan untuk melihat elemen paling atas tanpa menghapusnya.
- Alurnya jika:
  - Cek apakah stack kosong

    - Jika kosong maka throw error
    - Jika tidak
        - Kembalikan `arr[top]`.
- `top` selalu menunjuk elemen terakhir.
- Jadi cukup ambil tanpa mengubah struktur stack.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Pada code `bool Stack::isEmpty() {`

```
    return top == -1;
```

`}` terdapat Fungsi isEmpty() yang digunakan sebagai berikut:

- Fungsi ini digunakan untuk mengecek apakah stack kosong.
- Alurnya jika:
    - Badingkan nilai `top`.
    - Jika kosong maka throw error
    - Jika tidak
        - Kembalikan `arr[top]`.
        - Jika `-1` maka kosong.
        - Jika tidak maka ada isi.
- `top = -1` maka kosong.
- Setiap push maka naik.
- Setiap pop maka turun.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Jadi dari keseluruhan kode, dapat disimpulkan bahwa:

Implementasi stack menggunakan array dengan variabel top sebagai penunjuk elemen terakhit. Seluruh operasi pada stack memiliki kompleksitas waktu **O(1)**, karena setiap operasi hanya melibatkan akses langsung ke elemen teratas. Penggunaan *exception handling* bertujuan untuk menangani kondisi tidak valid seperti stack penuh (overflow) dan stack kosong (underflow), sehingga program menjadi lebih aman dan terkontrol.

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
    - Bilangan bulat
    - Operator: +, -, *, dan /.

Format Input:

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah.

Format Output:

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14

5 10 2 / 3 *	14

A. Source Code

Table 3. Source Code prob1.cpp

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    int n;
    cin >> n;

    stack<int> st;

    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;

        if (isdigit(s[0]) || (s.size() > 1)) {
            st.push(stoi(s));
        } else {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();

            if (s == "+") st.push(a + b);
            else if (s == "-") st.push(a - b);
            else if (s == "*") st.push(a * b);
            else if (s == "/") st.push(a / b);
        }
    }

    cout << st.top();
}
```

B. Output Program

Gambar 1. Screenshot Hasil Jawaban Soal 2A

C. Pembahasan

Program ini digunakan untuk menghitung ekpresi matematika dalam bentuk:

- Postfix (Reverse Polish Notation / RPN).

Pada code `int n;`

`cin >> n;`

`stack<int> st;` terdapat Inisialisasi yang digunakan sebagai berikut:

- `n` digunakan untuk jumlah simbol.
- `Stack<int> st` digunakan untuk jumlah simbol.

Pada code `for (int i = 0; i < n; i++) {`

`string s;`

`cin >> s;` terdapat Perulangan Input yang digunakan sebagai berikut:

- Membaca simbol satu per satu.
- Bisa berupa angka atau operator.

Pada code `if (isdigit(s[0]) || (s.size() > 1)) {`

`st.push(stoi(s));`

`}` terdapat fungsi Cek Apakah Angka yang digunakan sebagai berikut:

- `(isdigit(s[0])` digunakan untuk cek karakter pertama angka.
- `(s.size() > 1)` digunakan untuk angka negatif.
- `(stoi(s)` digunakan untuk ubah string jadi integer.

Pada code `else {`

`int b = st.top(); st.pop();`

`int a = st.top(); st.pop();` terdapat fungsi Jika Operator yang digunakan sebagai berikut:

- Ambil 2 angka terakhir di stack.
- `b` digunakan untuk angka terakhir.
- `a` digunakan untuk angka sebelum terakhir.

Pada code `if (s == "+") st.push(a + b);`

`else if (s == "-") st.push(a - b);`

`else if (s == "*") st.push(a * b);`

`else if (s == "/") st.push(a / b);` terdapat Proses Operasi yang digunakan sebagai berikut:

- Lakukan operasi sesuai simbol
- Hasilnya dimasukkan kembali ke stack.

Pada code `cout << st.top();` terdapat Output yang digunakan sebagai berikut:

- Setelah semua diproses, hanya tersisa 1 nilai
- Itu merupakan hasil akhirnya.

Alurnya jika:

- Badingkan nilai `top`.
- Jika kosong maka throw error
- Jika tidak
    - Kembalikan `arr[top]`.
    - Jika `-1` maka kosong.
    - Jika tidak maka ada isi.
- Stack selalu mengambil dua elemen terakhir.
- Operator selalu bekerja pada operand yang benar.
- Setelah operasi, hasil dikembalikan ke stack.
- Proses berulang sampai hanya tersisa satu nilai.
- Kompleksitas:
    - Waktu: **O(*n*)**
    - Ruang: **O(*n*)**.

Jadi dari keseluruhan kode, dapat disimpulkan bahwa:

Program ini menggunakan struktur data stack. Setiap bilangan disimpan ke dalam stack, sedangkan operator akan mengambil dua elemen terakhir untuk dilakukan operasi. Hasilnya kemudian dimasukkan kembali ke dalam stack. Metode ini efisien dengan kompleksitas waktu **O(*n*)** dan ruang **O(*n*)**.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 4. Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

class Queue
{
private:
    int arr[100];
    int front;
    int rear;
    int size;
public:
    Queue();
    void enqueue(int x);
    int dequeue();
    bool isEmpty();
};

#endif
```

Table 5. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

```

```
Queue::Queue()
{
    front = 0;
    rear = -1;
    size = 0;
}

void Queue::enqueue(int x)
{
    if (size == 100)
    {
        throw std::overflow_error("Queue penuh");
    }
    rear = (rear + 1) % 100;
    arr[rear] = x;
    size++;
}

int Queue::dequeue()
{
    if (isEmpty())
    {
        throw std::underflow_error("Queue kosong");
    }
    int val = arr[front];
    front = (front + 1) % 100;
    size--;
    return val;
}

bool Queue::isEmpty()
{
    return size == 0;
}
```

B. Pembahasan

Pada bagian Struktur Kelas *(queue.h)*

Bagian ini mendefinisikan cetak biru dari mesin antrean.

- Atribut:

    - `arr[100]` digunakan untuk array statis untuk menyimpan maksimal 100 elemen.
    - `front` digunakan untuk indeks untuk menunjuk elemen terdepan (yang akan keluar).
    - `rear` digunakan untuk indeks untuk menunjuk elemen terakhir yang dimasukkan.
    - `size` digunakan untuk menghitung jumlah elemen yang saat ini ada di dalam antrean.
- Kompleksitas:
    - Ruang: **O(1)** karena alokasi memori sudah tetap (statis).

Pada bagian Fungsi `enqueue(int x)` digunakan sebagai berikut:

- Fungsi ini digunakan untuk memasukkan data ke dalam antrean.
- Alurnya jika:
    - Cek apakah size sudah mencapai 100.
    - Jika `(size == 100`, maka throw overflow_error (antrean penuh).
    - Jika tidak
        - *Update* posisi `(rear)` menggunakan rumus `(rear + 1) % 100`.
    - Masukkan nilai *x* ke dalam `arr[rear]`.
    - Tambah nilai `size`.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Pada bagian Fungsi `dequeue()` digunakan sebagai berikut:

- Fungsi ini digunakan untuk mengambil dan menghapus elemen terdepan.

- Alurnya jika:
    - Panggil fungsi `isEmpty()` untuk mengecek ketersediaan data.
    - Jika kosong, maka throw overflow_error.
    - Jika ada isi:
        - Ambil nilai dari `arr[front]`.
        - *Update* posisi `(front)` menggunakan rumus `(front + 1) % 100`.
    - Kurangi nilai `size`.
    - Kembalikan (return) nilai yang diambil sebelumnya.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Pada bagian Fungsi `isEmpty()` digunakan sebagai berikut:

- Fungsi ini digunakan sebagai pelindung sebelum melakukan operasi pengambilan data.
- Alurnya jika:
    - Badingkan nilai `size` dengan 0.
    - Jika `size == 0` (True), maka antrean kosong.
    - Jika `size > 0` (False), maka antrean memiliki isi.
- Kompleksitas:
    - Waktu: **O(1)**
    - Ruang: **O(1)**.

Jadi dari keseluruhan kode, dapat disimpulkan bahwa:

Implementasi ini menggunakan metode *Circular Queue* dengan bantuan operator modulo (%). Hal ini memungkinkan penggunaan memori array secara efisien tanpa

harus menggeser elemen setiap kali data di ambil, sehingga performa tetap optimal dengan kompleksitas waktu **O(1)** untuk setiap operasinya.

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih. Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n jumlah elemen
- k ukuran jendela
- Ai elemen array

Format Input:

n k

A1 A2 ... An

Keterangan:

- Baris pertama *n* menyatakan jumlah atau panjang array. Lalu *k* besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol *A* sebanyak *n* buah.

Format Output:

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 6. Source Code prob2.cpp

```
#include "queue.h"
#include <iostream>
using namespace std;

int main()
{
    int n, k;
    cin >> n >> k;

    int arr[100];

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

```

```
    int sum = 0;

    for (int i = 0; i < k; i++)
    {
        sum += arr[i];
    }

    cout << sum;

    for (int i = k; i < n; i++)
    {
        sum += arr[i] - arr[i - k];
        cout << " " << sum;
    }

    return 0;
}
```

B. Output Program

Gambar 2. Screenshot Hasil Jawaban Soal 4A

C. Pembahasan

Pada bagian Inisialisasi Jendela Pertama digunakan sebagai berikut:

- Bagian ini adalah persiapan sebelum jendela mulai digeser.
- Alurnya jika:
    - Membaca input *n* (jumlah angka) dan *k* (ukuran jendela).
    - Menghitung jumlah total dari *k* elemen pertama (*A1* sampai *Ak*).
    - Hasil penjumlahan ini disimpan dalam variabel *sum* sebagai hasil untuk kelompok pertama.

- Kompleksitas:
    - Waktu: **O(*k*)**
    - Ruang: **O(*n*)** untuk menyimpan array.

Pada bagian Mekanisme Pergeseran Jendela digunakan sebagai berikut:

- Fungsi ini adalah inti dari efisiensi algoritma.
- Alurnya jika:
    - Loop berjalan dari indeks *k* sampai *n – 1*.
    - Tambah elemen baru yaitu menambahkan angka yang baru masuk ke sisi kanan jendela (`arr[i]`).
    - Kurangi elemen lama yaitu mengurangi angka yang ditinggalkan di sisi kiri jendela (`arr[i - k]`).
    - Cetak hasil `sum` yang baru.
- Kompleksitas:
    - Waktu: **O(*n*)**
    - Ruang: **O(*n*)** untuk menyimpan *n*.

Jadi dari keseluruhan kode, dapat disimpulkan bahwa:

Implementasi ini menggunakan teknik *Sliding Window*. Dengan memamfaatkan nilai penjumlahan sebelumnya, program hanya perlu melakukan dua operasi (tambah dan kurang) untuk setiap pergeseran, sehingga waktu eksekusi tetap stabil meskipun ukuran *k* sangat besar. Penggunaan array memungkinkan akses acak (random access) ke elemen lama yang harus dikurangi dengan cepat melalui indeks $i - k$.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-inipunyaCELI.git